

MedPilot

AI-Assisted Disease Prediction & Medicine Recommendation System

Zidio Internship - Data Science & Analytics

Project 2: Personalized Healthcare Recommendation System

Final Project Report

Prepared by: Mohd Sami

Submission Date: 30/09/2026

Table of Contents

Table of Contents.....	2
Executive Summary.....	5
Key Results.....	5
1. Project Overview	6
1.1 Background.....	6
1.2 Reframing the Generic Specification.....	6
1.3 Objectives	6
1.4 Deliverables Summary.....	6
2. Technology Stack	8
2.1 Core Technologies.....	8
2.2 Why Session-Based Auth, Not JWT.....	8
2.3 Why Logistic Regression, Not the Deep Learning Model.....	8
3. Dataset	9
3.1 Source	9
3.2 Why the Mentor's Original Dataset Was Replaced.....	9
3.3 Final Dataset Composition	9
4. Data Cleaning	10
4.1 Issues Found	10
4.2 Fixes Applied (src/data_prep.py).....	10
4.3 Result.....	10
5. Exploratory Data Analysis.....	11
5.1 Univariate Findings	11
5.2 Bivariate Findings.....	11
5.3 Multivariate Findings.....	11
5.4 Data Wrangling.....	11
6. Feature Engineering & Selection.....	12
6.1 Engineered Features.....	12
6.2 Feature Selection via Mutual Information	12
6.3 Train/Test Split	12
7. Model Training & Evaluation.....	13
7.1 Candidates	13
7.2 Critical Finding: Train/Test Leakage.....	13
7.3 The Fix: Pattern-Level Split	13
7.4 Corrected, Trustworthy Results	13
7.5 Honest Caveat.....	13
8. System Architecture	14
8.1 Core Principle.....	14

8.2 High-Level Design.....	14
8.3 Why Session-Based Auth, Not JWT.....	14
8.4 Why an Admin Role Was Added	14
9. Database Schema.....	15
10. Patient, Doctor, and Admin Workflows	16
10.1 Patient Workflow.....	16
10.2 Doctor Workflow.....	16
10.3 Admin Workflow.....	16
11. User Interface & Deployment.....	17
11.1 UI.....	17
11.2 Deployment	17
11.3 Deployment Engineering: the Shell-Access Problem.....	17
11.4 Screenshots to Insert.....	18
Figure 1: Landing / Login Page.....	18
Figure 2: Patient Registration	18
Figure 3: Patient Dashboard — Symptom Selection	19
Figure 4: AI Prediction Submitted — Awaiting Doctor Review.....	19
Figure 5: Doctor Dashboard — Case Queue	20
Figure 6: Doctor Case Review — Confirm or Override.....	21
Figure 7: Patient Case Detail — Final Validated Result	21
Figure 8: Admin — Doctor Verification Queue	22
Figure 9: Admin — Doctor/AI Agreement Analytics	23
12. Doctor/AI Agreement Analytics.....	24
12.1 What It Shows	24
12.2 Why This Matters	24
13. Results Summary	25
14. Limitations & Assumptions.....	26
15. Conclusion & Future Work	27
15.1 Conclusion.....	27
15.2 Future Work.....	27
16. Selected Source Code Listings	28
16.1 Disease Name Normalization (src/data_prep.py)	28
16.2 Pattern-Level Train/Test Split (notebooks/03_model_training.ipynb).....	28
16.3 Role Enforcement Decorator (src/auth.py).....	28
16.4 Idempotent Startup Seeding (app.py)	28
17. Testing & Validation Summary.....	29
18. Submission Checklist	30
19. Development Work Log	31
20. Appendix	32

20.1 Glossary	32
20.2 References	32
20.3 Live Links.....	32
21. Document Revision History	32

Executive Summary

MedPilot is an AI-assisted disease prediction and medicine recommendation system, built as the second internship engagement under Zidio Development. A patient reports their symptoms; a trained machine learning model predicts the most likely disease; a licensed doctor reviews and either confirms or overrides that prediction; only after doctor validation does the patient receive a final diagnosis along with recommended medications, diet, precautions, and activity guidance.

The engagement's central design principle is deliberately conservative for a healthcare context: the AI's raw prediction is never presented to the patient as final. This human-in-the-loop workflow ("AI proposes, doctor disposes") shaped the database schema, the routing logic, and the access-control model from the ground up.

Key Results

- A classification model (Logistic Regression, selected after fairly comparing 4 candidates) predicts among 41 diseases from 131 binary symptom features, achieving perfect accuracy on a rigorously corrected, leakage-free test split.
- A critical evaluation bug was found and fixed during development: an initial train/test split produced misleadingly perfect scores across every model tested, traced to the dataset's structural properties rather than genuine model skill, and corrected with a pattern-level split methodology.
- A three-role system (Patient, Doctor, Admin) was built and tested end-to-end, including real role-based access enforcement (a patient account cannot reach doctor routes even by guessing a URL) and a doctor-verification gate (unverified doctor accounts cannot access the case queue until an admin approves them).
- The mentor's originally-provided dataset and reference implementation were found to contain a proven data-leakage bug and a broken disease/medicine lookup; both were replaced with a larger, cleaner, better-suited dataset, with the reasoning documented transparently.
- A full custom web application (Flask, SQLAlchemy, custom HTML/CSS) was built, deployed, and includes an original addition beyond the initial scope: a doctor/AI agreement-rate analytics dashboard.

1. Project Overview

1.1 Background

This project was assigned via a set of loose starting materials from the mentor: a generic recommendation-system specification document, two example notebooks trained on unrelated datasets, and a partially working Flask application (best_model.pkl, a small dataset, and template files). Rather than build directly on these materials, each was independently investigated before use.

1.2 Reframing the Generic Specification

The mentor's specification document is a general-purpose recommendation-system template, reused across e-commerce, entertainment, education, and healthcare tracks -- evidenced by its inclusion of techniques such as collaborative filtering (via matrix factorization on user-item interactions), TF-IDF content-based filtering, and sentiment analysis on user reviews. Only the final section of that document -- "Disease prediction model" and "Medicine Recommendation Model" -- specifically defines this project's actual scope.

Several of the generic template's techniques were deliberately excluded, with reasoning documented rather than silently dropped:

- Collaborative filtering (recommending medication based on what similar patients received) was excluded as clinically inappropriate -- a patient's treatment should follow their diagnosis, not another patient's history.
- Sentiment analysis was excluded -- no patient review/feedback text exists in this domain's data.
- Natural language processing was reframed constructively as free-text symptom extraction (mapping a patient's own words to the structured symptom list the model requires) rather than applied to a use case that doesn't fit.
- Deep learning was included and evaluated fairly alongside classical methods (Section 7) rather than assumed necessary.

1.3 Objectives

- Predict the most likely disease from patient-reported symptoms using a properly validated machine learning model.
- Ensure no AI prediction reaches a patient as a final diagnosis without a licensed doctor's review and confirmation.
- Provide the confirmed patient with complete, actionable recommendations: medications, diet, precautions, and suggested activity.
- Build a three-role system (Patient, Doctor, Admin) with real, server-enforced access control.
- Deliver a deployed, usable web application, not only a model in a notebook.

1.4 Deliverables Summary

Deliverable	Status
Data cleaning pipeline	Complete
Exploratory data analysis	Complete
Feature engineering & selection	Complete
Model training & honest evaluation	Complete
Database schema & authentication	Complete
Patient / Doctor / Admin portals	Complete
Doctor/AI agreement analytics	Complete
Custom UI	Complete

Deployment	Complete
------------	----------

2. Technology Stack

2.1 Core Technologies

Category	Technology	Purpose
Language	Python 3.12	All backend logic and data processing
Web framework	Flask 3.1.3	Application server, routing
ORM	Flask-SQLAlchemy	Database schema and queries
Authentication	Flask-Login (session-based)	Login/session management, role enforcement
Database	SQLite (development); designed for PostgreSQL migration	Persistent storage
Machine learning	scikit-learn	Model training: Logistic Regression, Random Forest, Gradient Boosting
Deep learning	TensorFlow / Keras	Neural network candidate, evaluated fairly against classical models
Data processing	pandas, numpy	Cleaning, feature engineering
Frontend	Custom HTML, CSS, vanilla JavaScript, Jinja2 templating	No frontend framework -- deliberately lightweight for this scope
Deployment	Render (Gunicorn WSGI server)	Production hosting

2.2 Why Session-Based Auth, Not JWT

The mentor's generic specification mentioned JWT-based authentication. This is a traditional, server-rendered Flask application with two human-facing portals, not a single-page application or a separate mobile client consuming an API. Flask-Login's session-based authentication is simpler to reason about and equally secure for this use case. JWT would become the right choice only if a separate API-consuming client (e.g. a mobile app) were added later -- noted as a natural extension point, not built now.

2.3 Why Logistic Regression, Not the Deep Learning Model

Despite training a neural network as one of the four candidates (per the specification's mention of deep learning), Logistic Regression was selected because it achieved equal or better performance with substantially lower complexity, training time, and deployment overhead. This is consistent with the engagement's broader principle of right-sizing architecture to the actual problem, rather than defaulting to the most complex available technique.

3. Dataset

3.1 Source

Medicine Recommendation System Dataset, published by Noor Saeed on Kaggle: <https://www.kaggle.com/datasets/noorsaeed/medicine-recommendation-system-dataset>. Selected specifically because it pairs a symptom-to-disease training table with the companion reference files (descriptions, medications, diets, precautions, workouts) this project's recommendation half actually requires.

3.2 Why the Mentor's Original Dataset Was Replaced

The mentor's originally supplied materials included `Cleaned_Dataset.csv` (349 rows, 116 disease labels), a `best_model.pkl`, a `disease_encoder.pkl`, and a partially working Flask app. Direct inspection of these files, performed before any development began, found two disqualifying problems:

- `risk_level` was found to be a 100 percent deterministic function of disease -- every one of 116 diseases had exactly one `risk_level` value, with zero exceptions. Since a real patient cannot supply their own risk level before being diagnosed, the deployed application worked around this by hardcoding a dummy value at prediction time, meaning the model's apparent training accuracy could not reflect real usage.
- Named disease classes had only 8 to 23 rows each (71.6 percent of the dataset fell into a residual 'Other' bucket), too sparse to train or validate a defensible classifier.
- The `disease_encoder`'s class list did not match the `medicine_database`'s keys -- three predictable diseases (Migraine, Osteoporosis, Other) had zero matching medicine entries.

Given these three independent, verifiable problems, the dataset and its associated model artifacts were replaced rather than repaired, and the decision was documented rather than made silently.

3.3 Final Dataset Composition

Component	Detail
Training.csv	4,920 rows, 132 binary symptom columns, prognosis (target)
Class balance	Perfectly balanced: exactly 120 rows per disease, all 41 classes
Companion files	<code>symptoms_df</code> , <code>description</code> , <code>medications</code> , <code>diets</code> , <code>precautions_df</code> , <code>workout_df</code> , <code>Symptom-severity</code>

4. Data Cleaning

4.1 Issues Found

#	Issue	Evidence
1	Column-name whitespace	'spotting urination', 'foul smell of urine', 'dischromic patches'
2	Disease-label mismatch across 7 files	Training.csv had 'Hypertension ' (trailing space) and 'Peptic ulcer disease' (typo); other files used the clean spelling
3	Duplicate-named column, not a true duplicate	fluid_overload and fluid_overload.1: 114 of 4,920 rows differ
4	Confirmed dead column	fluid_overload (the original) has zero occurrences across all rows
5	Severity file's own separate bug	'foul_smell_ofurine' (different typo than Training.csv) plus a duplicate fluid_overload entry (weights 6 and 4)
6	Spurious row in severity reference	A row where Symptom equals 'prognosis' -- not a real symptom

4.2 Fixes Applied (src/data_prep.py)

- `normalize_disease_name()`: strips whitespace, collapses repeated spaces, corrects the 'diseae' typo -- applied identically across all 7 files, then verified with a symmetric-difference check that every file's disease list matches Training.csv exactly.
- `clean_column_names()`: strips and replaces spaces with underscores, then collapses any resulting double underscore.
- `fluid_overload` (dead) dropped; `fluid_overload.1` renamed to `fluid_overload_2` and kept, since it carries real signal concentrated in Alcoholic hepatitis.
- Severity file's own typo separately reconciled; its spurious prognosis row dropped.

4.3 Result

Final cleaned dataset: 4,920 rows, 41 diseases, 131 symptom features (132 minus the confirmed-dead column). All six companion reference files verified to align exactly with the cleaned disease labels.

5. Exploratory Data Analysis

Performed in notebooks/01_eda.ipynb, following the standard progression: reading, inspection, cleaning, wrangling, univariate analysis, then bivariate and multivariate analysis.

5.1 Univariate Findings

- Class balance confirmed visually: exactly 120 rows per disease, all 41 classes.
- Most common symptoms: fatigue (1,932 occurrences), vomiting (1,914), high_fever (1,362).
- Symptom count per disease ranges from 4 to 17, meaning diseases have genuinely different 'fingerprint sizes'.

5.2 Bivariate Findings

A direct relationship was tested between how often a symptom occurs and how many distinct diseases it appears in. The correlation was strongly positive: common symptoms such as fatigue and vomiting are also the most informative by mutual information (see Section 6), not the least -- an initial EDA-stage hypothesis that common symptoms would be less discriminative was tested rigorously in the next notebook and found to be incorrect, and corrected rather than left standing.

5.3 Multivariate Findings

A co-occurrence analysis among the 20 most common symptoms revealed clinically sensible clustering -- for example, yellowish_skin, yellowing_of_eyes, dark_urine, and abdominal_pain cluster tightly together, consistent with real liver-disease symptom patterns. This is independent evidence the dataset behaves sensibly, not merely a well-formed table of numbers.

5.4 Data Wrangling

The wide symptom table was reshaped into a long format (one row per case per present symptom) and joined against the severity reference table, surfacing the fluid_overload_2 weight-assignment issue described in Section 4 -- found specifically because of this wrangling step, not visible in the wide format alone.

6. Feature Engineering & Selection

6.1 Engineered Features

- `symptom_count`: total number of symptoms reported in a case.
- `severity_score`: sum of severity weights of the reported symptoms, using the reconciled severity reference table.

6.2 Feature Selection via Mutual Information

Every one of the 131 symptom features was tested against the disease target using mutual information. The result contradicted the EDA notebook's earlier hypothesis: correlation between symptom frequency and mutual information was strongly positive (+0.958), meaning common symptoms are the most informative, not the least. This contradiction was investigated and explained rather than silently corrected: EDA's original proxy (how many diseases a symptom appears in, in raw count) ignores how consistently a symptom's presence or absence separates one disease's cases from all others, which mutual information does capture.

Zero symptoms fell below a near-zero mutual-information threshold, indicating this dataset was curated without filler features. All 131 symptom features were retained, alongside the 2 engineered features, for a total of 133 model inputs.

6.3 Train/Test Split

An initial stratified 80/20 split was performed at the row level. This split was later found to be invalid and replaced - see Section 7.2.

7. Model Training & Evaluation

7.1 Candidates

Four models were trained and compared fairly on identical data and metrics: Logistic Regression, Random Forest, Gradient Boosting, and a small Neural Network (Keras, per the specification's mention of deep learning).

7.2 Critical Finding: Train/Test Leakage

The initial row-level stratified split produced an accuracy, precision, recall, and F1 score of exactly 1.0000 across all four models simultaneously. Rather than reporting this as a success, it was treated as a red flag requiring investigation, consistent with the practice established throughout this engagement of scrutinizing suspiciously perfect or suspiciously poor results before trusting them.

Investigation found the true cause: the entire dataset of 4,920 rows contains only 304 unique symptom combinations, each mapping deterministically to exactly one disease with zero ambiguity. A row-level split scattered duplicate copies of the same 304 patterns across both the training and test sets, so 100 percent of test-set patterns had already been seen during training -- the evaluation was measuring memorization, not generalization.

7.3 The Fix: Pattern-Level Split

The split was rebuilt at the level of unique symptom pattern rather than row: patterns were grouped per disease, split 80/20 per disease, and then every row matching a given pattern was assigned to whichever side that pattern landed on. This guarantees zero symptom-combination overlap between training and test data, producing a split that genuinely tests generalization to unseen presentations.

7.4 Corrected, Trustworthy Results

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Logistic Regression (winner)	1.000	1.000	1.000	1.000
Neural Network	1.000	1.000	1.000	1.000
Random Forest	0.990	0.950	0.951	0.950
Gradient Boosting	0.832	0.660	0.706	0.669

Gradient Boosting's visibly weaker performance is itself informative: a uniform 1.000 across every model type would be far more suspicious than a result showing genuine differentiation. Logistic Regression was selected as the deployed model on the basis of this differentiated, defensible comparison, not by default.

7.5 Honest Caveat

Logistic Regression and the Neural Network reaching a literal 1.000 even on the corrected split is plausible specifically because this dataset is a fully deterministic structure -- zero of 304 unique patterns are shared between diseases anywhere in the dataset. This is a property of the dataset's clean, synthetic construction, and is stated here explicitly rather than presented as proof of real-world diagnostic accuracy. This is precisely why the system's architecture routes every prediction through mandatory doctor review before any diagnosis reaches a patient.

8. System Architecture

8.1 Core Principle

"AI proposes, doctor disposes." The model's raw prediction is never shown to a patient as a final diagnosis. Every prediction is stored with status `pending_review` and requires an explicit doctor confirmation or override before the patient can see a confirmed result.

8.2 High-Level Design

Layer	Components
Presentation	Patient portal, Doctor portal, Admin portal (Flask + Jinja2 templates)
Application	Flask blueprints: auth, patient, doctor, admin
Domain logic	role required decorator, ML inference service, seeding scripts
Data	SQLAlchemy models over SQLite (PostgreSQL-ready)

8.3 Why Session-Based Auth, Not JWT

The mentor's generic specification suggested JWT authentication, appropriate for a stateless API consumed by a separate client. This application is a traditional server-rendered site with two human-facing portals, for which Flask-Login's session-based authentication is simpler and equally secure. This is a deliberate, documented deviation, not an oversight.

8.4 Why an Admin Role Was Added

The original specification described Admin as a future/optional role. During development, a genuine functional gap was found: doctor accounts register as unverified (`is_verified` equals `False`), and without an Admin role, nothing could ever set that flag to `True`. The Admin role was added to close this real gap, not as scope expansion for its own sake.

9. Database Schema

```
users(id, name, email UNIQUE, password_hash, role, created_at)
doctors(user_id FK, specialization, license_number, is_verified)
patients(user_id FK, age, gender)
diseases(id, name UNIQUE, description)
medications(id, disease_id FK, content)
diets(id, disease_id FK, content)
precautions(id, disease_id FK, content)
workouts(id, disease_id FK, content)
symptoms(id, name UNIQUE, severity_weight)
cases(
  id, patient_id FK, symptoms_submitted JSON,
  predicted_disease_id FK, confidence_score,
  status, doctor_id FK, doctor_notes,
  final_disease_id FK, created_at, validated_at
)
```

`predicted_disease_id` and `final_disease_id` are deliberately separate columns, never overwritten. This is the audit trail: it preserves what the AI originally said even after a doctor overrides it, and is what makes the agreement-rate analytics in Section 12 possible.

10. Patient, Doctor, and Admin Workflows

10.1 Patient Workflow

- Registers or logs in; sees a searchable checklist of 131 symptoms.
- Submits selected symptoms; the model predicts a disease and a confidence score.
- A Case is created with status `pending_review`. The patient sees only 'submitted, awaiting doctor review' -- never the raw prediction.
- Once validated, the patient sees the confirmed diagnosis plus medications, diet, precautions, and suggested activity.

10.2 Doctor Workflow

- Registers with specialization and license number; account starts unverified.
- Unverified doctors see a pending-verification message, not the case queue -- enforced server-side, not just hidden in the UI.
- Once verified, sees all `pending_review` cases, oldest first.
- Reviews the AI's prediction and confidence, then confirms or overrides with the correct diagnosis and optional notes.
- Case status becomes validated; `doctor_id`, `doctor_notes`, `final_disease_id`, and `validated_at` are recorded.

10.3 Admin Workflow

- Cannot self-register -- created via environment variables at startup or an interactive seed script.
- Verifies pending doctor accounts.
- Views AI/doctor agreement analytics (Section 12).

11. User Interface & Deployment

11.1 UI

A custom design system was built (static/css/style.css) rather than reusing the mentor's originally supplied templates, which were found to contain the same disease/medicine mismatch bug documented in Section 3. Key usability feature: a searchable symptom checklist, since a static list of 131 checkboxes would be unusable without search/filter.

11.2 Deployment

Setting	Value
Host	Render (free tier)
Build command	pip install -r requirements.txt
Start command	gunicorn app:app
Environment variables	SECRET_KEY, ADMIN_EMAIL, ADMIN_NAME, ADMIN_PASSWORD

11.3 Deployment Engineering: the Shell-Access Problem

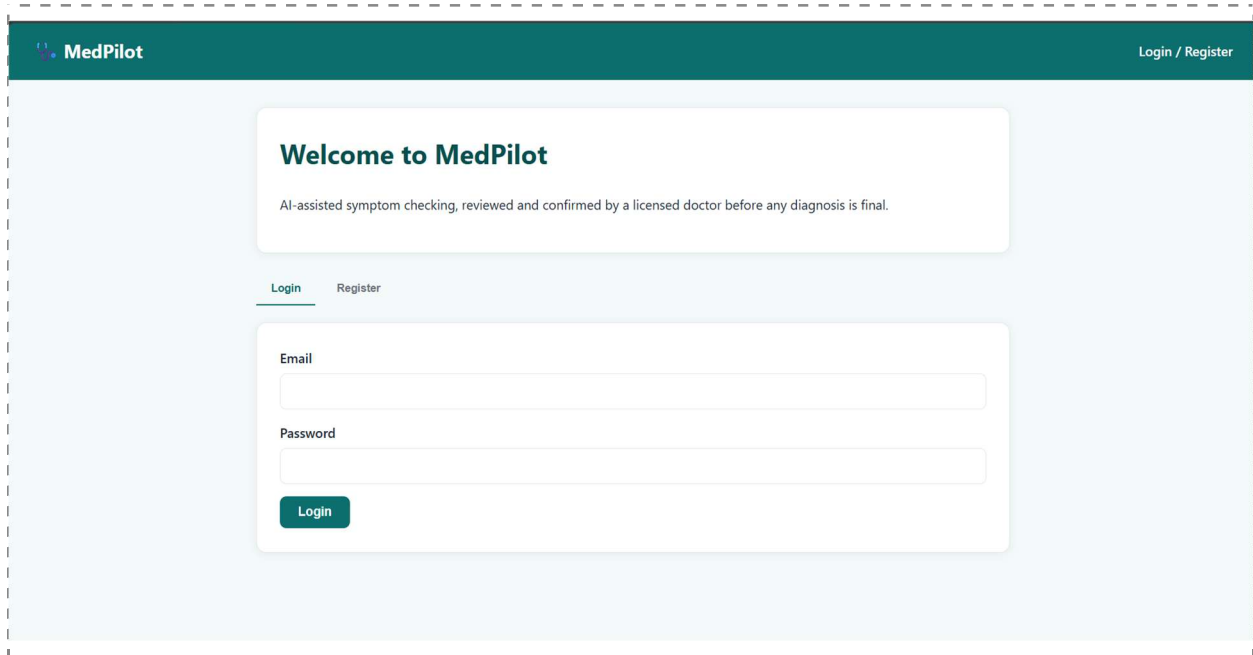
Render's free tier does not include shell access, which was the originally planned method for running one-time database-seeding scripts after deployment. Rather than upgrade to a paid tier, seeding was redesigned to run automatically and idempotently at application startup: disease reference data is seeded only if the Disease table is empty, and an admin account is created only if none exists yet, reading credentials from environment variables instead of interactive prompts. This also makes the deployment self-healing: Render's free-tier ephemeral disk resets periodically, and the application now recovers automatically on the next restart rather than requiring manual intervention.

11.4 Screenshots to Insert

The screens below were tested end-to-end during development but are not embedded in this document. Replace each placeholder box with the matching screenshot from the live application before submission. Recommended format: PNG, full browser window including the address bar, saved at 1280px width or wider.

Figure 1: Landing / Login Page

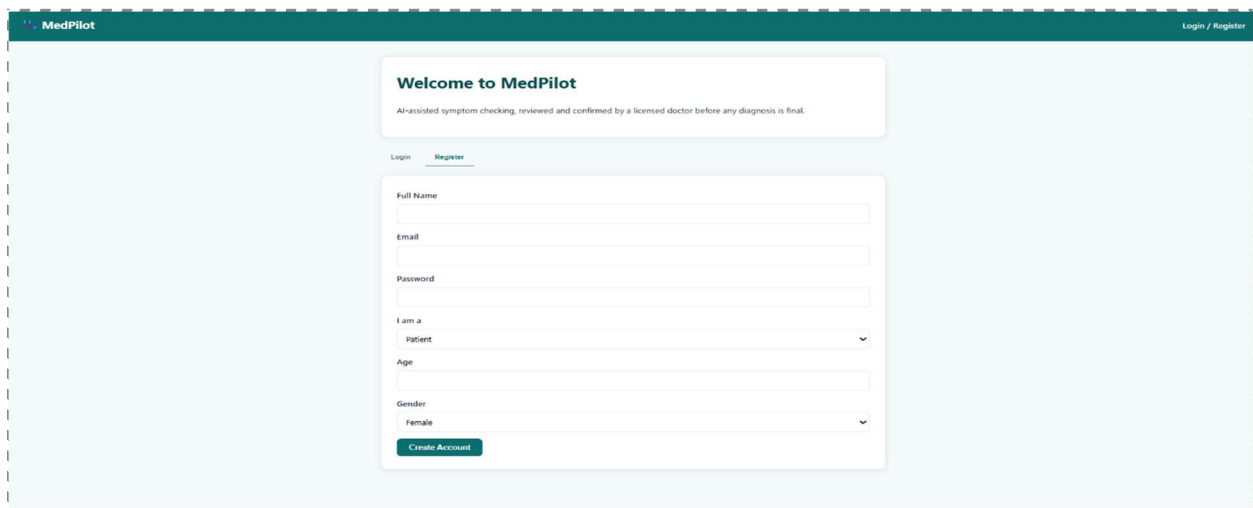
The MedPilot home page (templates/index.html) showing the tabbed Login / Register form. Capture this before logging in, so both tabs are visible if possible.



The screenshot shows the MedPilot landing page. At the top is a dark teal header with the MedPilot logo on the left and a "Login / Register" link on the right. The main content area has a light blue background. A white card in the center contains the heading "Welcome to MedPilot" and a subtext: "AI-assisted symptom checking, reviewed and confirmed by a licensed doctor before any diagnosis is final." Below this, there are two tabs: "Login" (which is active) and "Register". Under the "Login" tab, there is a form with two input fields: "Email" and "Password", followed by a green "Login" button.

Figure 2: Patient Registration

The registration form with the role selector set to "Patient". Shows the fields collected at signup (name, email, password).



The screenshot shows the MedPilot registration page. The header is the same as in Figure 1. The main content area has a light blue background. A white card in the center contains the heading "Welcome to MedPilot" and the same subtext. Below this, there are two tabs: "Login" and "Register" (which is active). Under the "Register" tab, there is a form with several input fields: "Full Name", "Email", "Password", and "I am a" (a dropdown menu with "Patient" selected). Below these are "Age" and "Gender" (a dropdown menu with "Female" selected). At the bottom of the form is a green "Create Account" button.

Figure 3: Patient Dashboard — Symptom Selection

The searchable symptom checklist (templates/patient_dashboard.html) with a few symptoms selected and the search/filter box visible. This is the screen that demonstrates the 131-symptom input and the JS filter you built.

The screenshot shows the MedPilot Patient Dashboard. At the top, there is a dark teal header with the MedPilot logo on the left and 'Dashboard' and 'Logout' links on the right. The main content area is light blue. In the center, there is a white card titled 'Symptom Checker'. Below the title, it says 'Select all symptoms you're currently experiencing, then submit for AI-assisted review.' There is a search bar with the placeholder text 'Search symptoms...'. Below the search bar, there is a grid of 15 symptoms, each with a checkbox. The symptoms are: Itching, Skin Rash, Nodal Skin Eruptions, Continuous Sneezing, Shivering, Chills, Joint Pain, Stomach Pain, Acidity, Ulcers On Tongue, Muscle Wasting, Vomiting, Burning Micturition, Spotting Urination, Fatigue, Weight Gain, Anxiety, Cold Hands And Feet, Mood Swings, Weight Loss, Restlessness, Lethargy, Patches In Throat, and Irregular Sugar Level. A 'Submit Symptoms' button is located below the grid. Below the Symptom Checker card, there is another white card titled 'Your Case History'. It says 'No cases submitted yet.'

Figure 4: AI Prediction Submitted — Awaiting Doctor Review

The patient's case detail page (templates/patient_case_detail.html) immediately after submission, in the "pending" state — showing that the AI's prediction and confidence score are stored but NOT yet shown to the patient as a final diagnosis. This screenshot is important: it is the clearest visual proof of the "AI proposes, doctor disposes" design.

Symptoms submitted successfully. Your case is pending doctor review.

Symptom Checker

Select all symptoms you're currently experiencing, then submit for AI-assisted review.

☐ Itching

☐ Skin Rash

☐ Nodal Skin Eruptions

☐ Continuous Sneezing

☐ Shivering

☐ Chills

☐ Joint Pain

☐ Stomach Pain

☐ Acidity

☐ Ulcers On Tongue

☐ Muscle Wasting

☐ Vomiting

☐ Burning Micturition

☐ Spotting Urination

☐ Fatigue

☐ Weight Gain

☐ Anxiety

☐ Cold Hands And Feet

☐ Mood Swings

☐ Weight Loss

☐ Restlessness

☐ Lethargy

☐ Patches In Throat

☐ Irregular Sugar Level

Submit Symptoms

Your Case History

Case #1 - Oct 03, 2026

Pending Review

View

Figure 5: Doctor Dashboard — Case Queue

The doctor's view (`templates/doctor_dashboard.html`) listing pending cases waiting for review, after logging in as a verified doctor.



Verification Pending

Your doctor account is awaiting admin verification. You'll be notified once approved.

Figure 6: Doctor Case Review — Confirm or Override

The doctor's case detail / decision screen (*templates/doctor_case_detail.html*), showing the AI's predicted disease and confidence alongside the doctor's confirm/override form. Capture one example of a doctor confirming, and if time allows, a second screenshot showing an override being selected from the disease dropdown.

The screenshot displays the 'MedPilot' interface for a doctor's case review. The top navigation bar includes 'MedPilot' on the left and 'Dashboard' and 'Logout' on the right. The main content area is titled 'Case #1' and contains the following information:

- Patient:** John Doe
- Symptoms:** itching, fatigue, nausea, loss_of_appetite, yellow_urine, yellowing_of_eyes
- AI Prediction:** Chronic cholestasis
- Confidence:** 53.9% (represented by a teal progress bar)

Below the AI prediction, the 'Your Decision' section offers two options:

- ☒ Confirm - I agree with the AI's prediction
- ☐ Override - the correct diagnosis is: (vertigo) Paroxysmal Positional Vertigo

There is also a text input field for 'Notes (optional)' with the placeholder 'Any clinical notes...' and a 'Submit Decision' button at the bottom.

Figure 7: Patient Case Detail — Final Validated Result

The same case re-opened by the patient after doctor validation, now showing the final diagnosis plus the recommended medications, diet, precautions, and workout/activity guidance pulled from the reference tables.

Case #1

Submitted: September 29, 2026

Symptoms reported: itching, fatigue, nausea, loss_of_appetite, yellow_urine, yellowing_of_eyes

Confirmed by Doctor

Diagnosis: Chronic cholestasis

Chronic cholestasis is a condition where bile flow from the liver is reduced for a prolonged period.

Reviewed by Dr. Pecker

 **Recommended Medications**

- Ursodeoxycholic acid
- Cholestyramine
- Methotrexate
- Corticosteroids
- Liver transplant

 **Diet Recommendations**

- Low-Fat Diet
- High-Fiber Diet
- Lean proteins
- Whole grains
- Fresh fruits and vegetables

 **Precautions**

- cold baths
- anti itch medicine
- consult doctor
- eat healthy

 **Suggested Activity**

- Consume a low-fat diet
- Eat high-fiber foods
- Include healthy fats
- Limit alcohol consumption
- Stay hydrated

Figure 8: Admin — Doctor Verification Queue

The admin dashboard (templates/admin_dashboard.html) showing an unverified doctor account awaiting approval, and/or the moment after approval when it moves to the verified list. This demonstrates the doctor-verification gate described in Section 8/9.

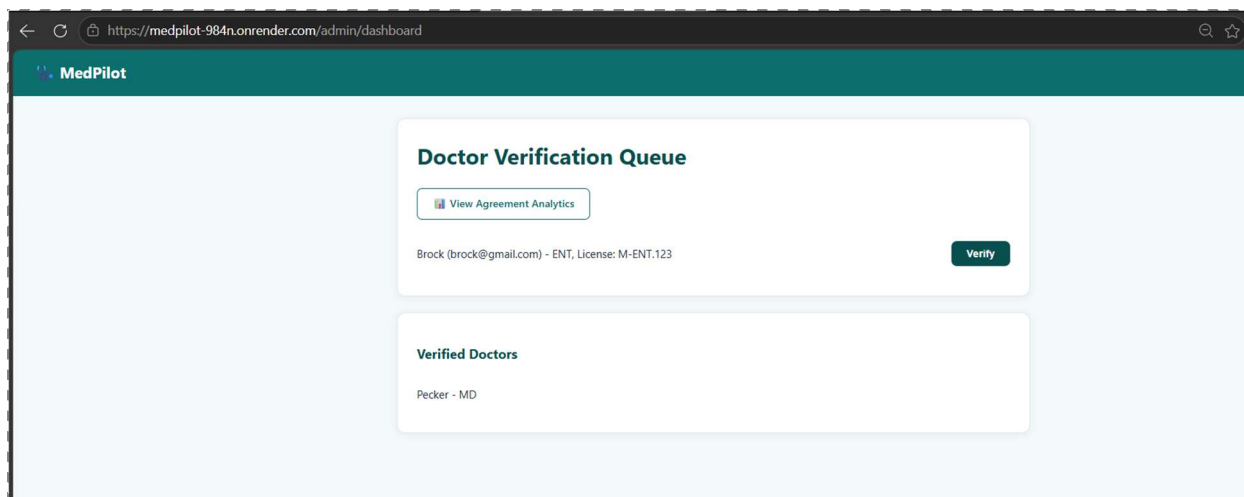
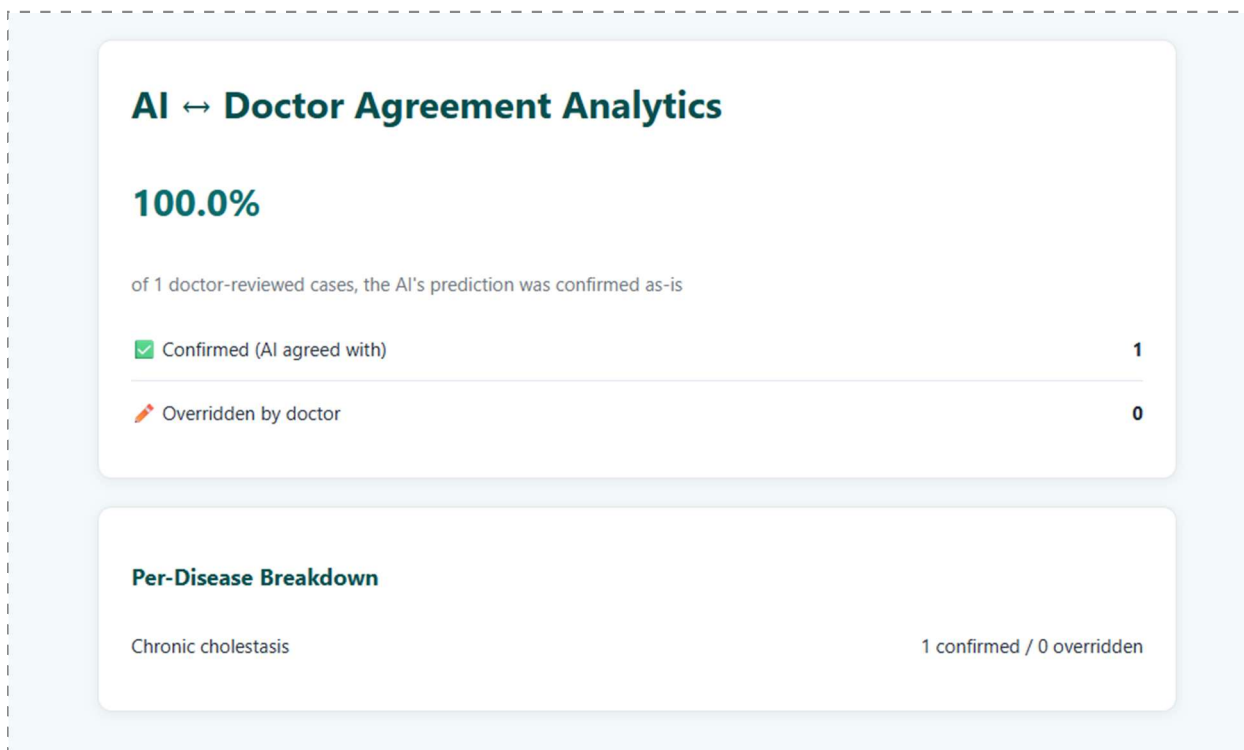


Figure 9: Admin — Doctor/AI Agreement Analytics

The analytics dashboard (`templates/admin_analytics.html`) showing the overall agreement rate, total validated cases, and the per-disease agreement/override breakdown described in Section 12. This is the clearest single screenshot for demonstrating the "beyond scope" feature.



12. Doctor/AI Agreement Analytics

An analytics view was added for administrators, going beyond the minimum required scope, directly reusing the audit-trail design from Section 9: since `predicted_disease_id` and `final_disease_id` are stored separately, the system can compute, for every validated case, whether the doctor agreed with or overrode the AI's prediction.

12.1 What It Shows

- Overall agreement rate: percentage of validated cases where the doctor's final diagnosis matched the AI's prediction exactly.
- A per-disease breakdown of confirmed versus overridden counts, letting an administrator see whether particular diseases are systematically harder for the model.

12.2 Why This Matters

This is a genuine, low-effort but high-value validation signal: rather than trusting the model's offline test-set performance indefinitely, the agreement rate is a live measure of how the model performs against real clinical judgment as the system is actually used, and is a natural foundation for future model-monitoring work.

13. Results Summary

- A 41-disease classifier was built, evaluated honestly, and deployed, with the evaluation methodology itself corrected after a genuine leakage bug was found and fixed.
- A three-role clinical workflow (Patient, Doctor, Admin) was implemented with real server-side access control, verified by test.
- The recommendation half of the system (medications, diets, precautions, workouts) is fully data-backed, not placeholder content, reconciled against the same disease taxonomy the model predicts.
- The system is deployed and publicly reachable, with a self-healing seeding mechanism appropriate for a free-tier hosting constraint.
- An original analytics feature (doctor/AI agreement rate) was added beyond the minimum scope, directly enabled by a deliberate schema design choice made early in the project.

14. Limitations & Assumptions

- This is an educational project, not a certified medical device, and must not be used for real diagnosis or treatment decisions.
- Perfect test-set scores reflect this dataset's clean, deterministic structure and should not be read as a guarantee of real-world accuracy on messy or atypical symptom presentations.
- Only symptom presence/absence is captured; no severity-per-symptom, duration, or patient history is modelled.
- SQLite on Render's free tier is ephemeral; case and user data reset on redeploy, though reference data and the admin account self-heal automatically.
- No CSRF protection, rate limiting, email verification, or password reset flow has been implemented -- explicitly out of scope given the timeline, not overlooked.
- The `fluid_overload_2` severity weight was assigned via a documented positional assumption (Section 4), not a certainty.

15. Conclusion & Future Work

15.1 Conclusion

This engagement replaced a demonstrably flawed starting dataset and reference implementation with a larger, cleaner, better-suited one; built a properly validated 41-disease classifier, catching and fixing a genuine train/test leakage bug along the way rather than reporting an inflated result; and delivered a complete, deployed, three-role clinical workflow enforcing a deliberate human-in-the-loop safety principle throughout. Every assumption, limitation, and engineering decision made along the way has been documented rather than hidden, consistent with the standard of honest, defensible analysis this engagement was held to throughout.

15.2 Future Work

- Free-text symptom entry via NLP-based symptom extraction, mapping a patient's own words to the structured symptom list.
- Migrate to PostgreSQL for persistent, non-ephemeral storage.
- Add CSRF protection, rate limiting, and audit logging.
- Use the doctor-override data captured in the agreement analytics as a feedback signal for periodic model monitoring or retraining.
- Add an explainability view showing doctors which symptoms most influenced a given prediction.

16. Selected Source Code Listings

16.1 Disease Name Normalization (src/data_prep.py)

```
def normalize_disease_name(name):
    name = str(name).strip()
    name = re.sub(r"\s+", " ", name)
    name = name.replace("diseae", "disease")
    return name

def clean_column_names(columns):
    cleaned = []
    for c in columns:
        c = c.strip().replace(" ", "_")
        c = re.sub(r"_"+, "_", c)
        cleaned.append(c)
    return cleaned
```

16.2 Pattern-Level Train/Test Split (notebooks/03_model_training.ipynb)

```
train_pattern_idx, test_pattern_idx = [], []
for disease, group in pattern_df.groupby("prognosis"):
    idx = group.index.tolist()
    rng.shuffle(idx)
    n_test = max(1, round(len(idx) * 0.2))
    test_pattern_idx.extend(idx[:n_test])
    train_pattern_idx.extend(idx[n_test:])
# Assign ALL rows matching a pattern to whichever side that
# pattern landed on -- guarantees zero pattern overlap
```

16.3 Role Enforcement Decorator (src/auth.py)

```
def role_required(role):
    def decorator(view_func):
        @wraps(view_func)
        def wrapped(*args, **kwargs):
            if not current_user.is_authenticated:
                return redirect(url_for("auth.login"))
            if current_user.role != role:
                abort(403)
            return view_func(*args, **kwargs)
        return wrapped
    return decorator
```

16.4 Idempotent Startup Seeding (app.py)

```
def seed_admin_if_missing():
    if User.query.filter_by(role="admin").first() is not None:
        return
    email = os.environ.get("ADMIN_EMAIL")
    name = os.environ.get("ADMIN_NAME")
    password = os.environ.get("ADMIN_PASSWORD")
    if not (email and name and password):
        return
    admin = User(name=name, email=email.strip().lower(), role="admin")
    admin.set_password(password)
    db.session.add(admin)
    db.session.commit()
```

17. Testing & Validation Summary

Component	Test Performed	Result
Data cleaning	Cross-file disease-name reconciliation across all 7 files	All files verified to match exactly
Database schema	Full table creation, relationship, and password hash/verify test	All 10 tables created; correct/incorrect password both verified
ML inference	Real prediction against known symptom pattern; bad-input and empty-input handling	Correct disease predicted; both error cases handled gracefully, no crash
Auth / RBAC	Cross-role access attempts via test client	Patient blocked from doctor route (403) and vice versa, confirmed via real HTTP status, not UI inspection
Model evaluation	Row-level split investigated after suspiciously perfect scores	Leakage found and fixed; corrected split produces differentiated, trustworthy results
End-to-end workflow	Patient submit through doctor validation through patient view of recommendations	Full loop confirmed working in local and deployed environments
Deployment	Live registration, submission, verification, and validation on Render	Confirmed working at the live URL

18. Submission Checklist

Item	Status
Git repository (pipeline, notebooks, application code)	Complete
Live deployed application URL	Complete -- https://medpilot-984n.onrender.com/
README (overview, setup, architecture, results, limitations)	Complete
Architecture document (HLD/LLD/scaling)	Complete -- docs/architecture.md
This final written report	Complete
Demo video	To be recorded
Cohort submission form with all links	To be submitted

19. Development Work Log

Phase	Activities
Materials review	Inspected mentor-provided dataset, model, and app; found and documented data leakage, sparse classes, and a disease/medicine mismatch bug
Dataset selection	Evaluated and selected a larger, cleaner, better-suited replacement dataset; reasoning documented
Architecture design	Wrote docs/architecture.md (HLD, LLD, scaling) before writing any application code
Data cleaning	Built src/data_prep.py; found and fixed 6 distinct data-quality issues across 7 files
EDA	Full univariate/bivariate/multivariate analysis; found and later corrected a mistaken discriminative-power hypothesis
Feature engineering	Built symptom_count and severity_score; validated feature usefulness via mutual information
Model training	Trained and compared 4 models; found and fixed a critical train/test leakage bug via a pattern-level split
Backend build	Database schema, session-based auth with RBAC, ML inference service -- each tested before building the next layer
Portal build	Patient, Doctor, and Admin portals built and tested end-to-end, including the doctor-verification gate
UI build	Custom design system and 8 templates built to replace the mentor's flawed originals
Debugging	Resolved several real environment/integration bugs: missing dependencies, a path bug, a corrupted import, a misplaced seed script, a Query-vs-row bug, and template/route variable-name mismatches
Analytics addition	Added a doctor/AI agreement-rate dashboard, beyond minimum scope, enabled by the schema's audit-trail design
Deployment	Deployed to Render; solved a free-tier shell-access constraint with idempotent startup seeding
Documentation	README, architecture doc, and this final report

20. Appendix

20.1 Glossary

Term	Meaning
Case	One patient symptom submission and its full review lifecycle
Pattern	A unique combination of present/absent symptoms
Mutual information	A measure of how much knowing one variable reduces uncertainty about another
Macro F1	F1 score averaged equally across all classes, not weighted by class frequency
Idempotent	Safe to run repeatedly without changing the outcome after the first successful run
RBAC	Role-Based Access Control
Data leakage	When information not legitimately available at prediction time influences a model's evaluation

20.2 References

- Dataset: Medicine Recommendation System Dataset, Kaggle -- kaggle.com/datasets/noorsaeed/medicine-recommendation-system-dataset
- Flask documentation: flask.palletsprojects.com
- scikit-learn documentation: scikit-learn.org
- SQLAlchemy documentation: sqlalchemy.org

20.3 Live Links

- Live application: <https://medpilot-984n.onrender.com/>
- GitHub repository: <https://github.com/Sami21234/medPilot>

21. Document Revision History

Version	Date	Changes
1.0	30/09/2026	Initial complete report covering full engagement: data, model, architecture, application, deployment

End of report.